算子 Batch 与 Single 执行差异验证报告

首期 8 个算子 · 三条执行路径 · 多序列长度 × 多轮重复

510x	164x	75x	0.12x
zip_concat 加速比（L=3000）	count_distinct 加速比（L=3000）	calc_delta_seq 加速比（L=3000）	log_base 加速比（批开销反噬）

一、验证环境与方法

日期	2026-08-12
机器	Windows 10.0.26200 x86_64（Git Bash / MINGW64）
Java	OpenJDK 21.0.12（Temurin LTS）
工作负载	8 组请求 × 每组 1000 候选行 = 8000 行；序列长度分 50 / 200 / 1000 / 3000 四档
统计口径	单次运行内 5 个测量轮次取 median；每个配置重复 3 次，再取 median-of-medians

三条执行路径均直接调用 OperatorRegistry，不含调度与列物化开销：

- single（不走 Batch）：外层循环逐行调 SingleOperatorKernel，每行全量重算；
- batchScalar（走 Batch 载体）：由 SingleLoopBatchOperatorKernel 逐行适配；
- batchNative（走原生 Batch）：BatchOperatorKernel 按（group, sequence, 参数）身份键批内复用。

二、共享序列场景（组内候选行共享请求序列，批内按 identity 复用）

模拟在线请求：同一请求组内的候选行共享同一条请求序列对象（与真实引擎一致），原生 Batch 的批内缓存按 identity 命中。表中时间为 median-of-medians（单位 ms）。

共享 · 序列长度 = 50

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	7.254	4.796	2.333	3.11x	1.51x
count_distinct	8.866	8.051	2.128	4.17x	1.10x
discrete	4.517	3.631	3.293	1.37x	1.24x
find_indices	2.444	2.279	3.107	0.79x	1.07x
get_seq_length	0.286	0.232	0.965	0.30x	1.23x
log_base	0.489	0.523	4.143	0.12x	0.93x
slice_by_indices	0.853	0.840	2.814	0.30x	1.02x
zip_concat	15.565	13.948	0.659	23.62x	1.12x

共享 · 序列长度 = 200

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	17.834	18.592	2.576	6.92x	0.96x
count_distinct	28.318	26.978	2.049	13.82x	1.05x
discrete	3.592	3.243	3.228	1.11x	1.11x
find_indices	9.040	7.292	3.118	2.90x	1.24x
get_seq_length	0.199	0.164	0.900	0.22x	1.21x
log_base	0.443	0.398	3.183	0.14x	1.11x
slice_by_indices	0.709	0.766	2.549	0.28x	0.93x
zip_concat	60.644	51.986	0.647	93.73x	1.17x

共享 · 序列长度 = 1000

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	231.161	203.302	4.295	53.82x	1.14x
count_distinct	126.371	124.981	2.240	56.42x	1.01x
discrete	4.181	4.312	4.112	1.02x	0.97x
find_indices	48.330	48.416	4.163	11.61x	1.00x
get_seq_length	0.235	0.244	1.081	0.22x	0.96x
log_base	0.509	0.623	3.567	0.14x	0.82x
slice_by_indices	0.686	0.630	2.584	0.27x	1.09x
zip_concat	466.262	488.793	1.559	299.08x	0.95x

共享 · 序列长度 = 3000

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	710.621	733.788	9.474	75.01x	0.97x
count_distinct	429.785	371.918	2.625	163.73x	1.16x
discrete	4.071	4.308	4.173	0.98x	0.94x
find_indices	162.378	147.878	4.856	33.44x	1.10x
get_seq_length	0.240	0.236	0.983	0.24x	1.02x
log_base	0.683	0.702	4.169	0.16x	0.97x
slice_by_indices	1.080	0.963	2.850	0.38x	1.12x
zip_concat	1519.593	1728.800	2.980	509.93x	0.88x

三、独立参数场景（每行独立参数，批内无复用）

每行参数独立（序列对象与参数均不共享），批内 identity 缓存全部失效，用于观察无复用场景下 Batch 通路的真实开销。

独立参数 · 序列长度 = 200

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	21.448	25.498	27.420	0.78x	0.84x
count_distinct	36.936	38.219	41.350	0.89x	0.97x
discrete	5.056	4.539	9.950	0.51x	1.11x
find_indices	9.650	8.152	116.312	0.08x	1.18x
get_seq_length	0.617	0.368	1.224	0.50x	1.68x
log_base	0.611	0.665	5.137	0.12x	0.92x
slice_by_indices	1.434	1.283	5.848	0.25x	1.12x
zip_concat	78.980	78.276	85.493	0.92x	1.01x

独立参数 · 序列长度 = 1000

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	322.595	288.786	288.386	1.12x	1.12x
count_distinct	227.953	233.523	230.330	0.99x	0.98x
discrete	8.096	7.268	13.720	0.59x	1.11x
find_indices	63.721	59.297	428.719	0.15x	1.07x
get_seq_length	0.976	0.796	1.905	0.51x	1.23x
log_base	0.772	0.813	9.116	0.08x	0.95x
slice_by_indices	2.081	2.342	7.933	0.26x	0.89x
zip_concat	758.797	760.510	817.804	0.93x	1.00x

四、双序列特征场景（每组请求携带两个共享序列特征 seqA + seqB）

每组请求携带两个共享原始序列特征：seqA（字符串行为序列）与 seqB（数值序列），候选行共享同一组对象；zip_concat 直接拼接 seqA+seqB，get_seq_length / discrete / log_base 取 seqB，其余算子取 seqA。批内复用键同时覆盖两个序列特征，如 zip_concat 的 (group, seqA, seqB)。

共享 · 双序列 · 序列长度 = 50

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	6.688	5.384	2.317	2.89x	1.24x
count_distinct	10.048	8.371	2.031	4.95x	1.20x
discrete	3.882	3.837	3.504	1.11x	1.01x
find_indices	3.378	2.674	3.650	0.93x	1.26x
get_seq_length	0.243	0.169	1.087	0.22x	1.44x
log_base	0.438	0.478	3.480	0.13x	0.92x
slice_by_indices	0.727	0.756	2.757	0.26x	0.96x
zip_concat	36.649	38.016	0.642	57.09x	0.96x

共享 · 双序列 · 序列长度 = 200

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	23.078	22.917	2.756	8.37x	1.01x
count_distinct	32.089	37.747	2.501	12.83x	0.85x
discrete	3.179	3.111	3.372	0.94x	1.02x
find_indices	8.093	7.331	3.320	2.44x	1.10x
get_seq_length	0.228	0.416	0.936	0.24x	0.55x
log_base	0.446	0.487	3.894	0.11x	0.92x
slice_by_indices	0.665	0.706	2.614	0.25x	0.94x
zip_concat	172.860	162.329	0.756	228.65x	1.06x

共享 · 双序列 · 序列长度 = 1000

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	216.016	199.755	3.997	54.04x	1.08x
count_distinct	161.714	159.192	2.240	72.19x	1.02x
discrete	4.306	3.754	3.586	1.20x	1.15x
find_indices	44.764	48.546	5.256	8.52x	0.92x
get_seq_length	0.277	0.240	1.193	0.23x	1.15x
log_base	0.493	0.562	3.530	0.14x	0.88x
slice_by_indices	0.715	0.615	2.515	0.28x	1.16x
zip_concat	948.596	1076.292	1.909	496.91x	0.88x

共享 · 双序列 · 序列长度 = 3000

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	938.769	1002.090	10.711	87.65x	0.94x
count_distinct	630.702	612.130	3.501	180.15x	1.03x
discrete	6.661	6.172	5.720	1.16x	1.08x
find_indices	194.895	196.801	6.311	30.88x	0.99x
get_seq_length	0.349	0.279	1.649	0.21x	1.25x
log_base	0.703	1.093	6.537	0.11x	0.64x
slice_by_indices	1.121	1.240	4.151	0.27x	0.90x
zip_concat	3691.158	3888.265	5.177	712.99x	0.95x

独立参数 · 双序列 · 序列长度 = 200

算子	single (ms)	batchScalar (ms)	batchNative (ms)	native/single	scalar/single
calc_delta_seq	20.138	20.334	27.203	0.74x	0.99x
count_distinct	36.468	36.357	41.387	0.88x	1.00x
discrete	4.859	5.011	8.985	0.54x	0.97x
find_indices	9.314	7.545	111.007	0.08x	1.23x
get_seq_length	0.720	0.375	1.258	0.57x	1.92x
log_base	0.552	0.614	5.769	0.10x	0.90x
slice_by_indices	1.120	0.948	4.741	0.24x	1.18x
zip_concat	168.930	175.733	166.896	1.01x	0.96x

五、特征表达式层验证（8 个算子各一 DERIVED 特征表达式）

8 个算子各以一个 DERIVED 特征表达式定义，走完整引擎链路（表达式解析 → 逻辑 DAG → 物理计划 → runtime 分派）。候选特征（USER×ITEM）：bucket_level=discrete(amount, [0,10,50,100,500])、log_amount=log_base(amount, 2, 1048576)、target_positions=find_indices(codes, target_tag)、delta_sequence=calc_delta_seq(numbers, delta_base)；共享序列特征（USER）：code_window=slice_by_indices(codes, [0..10])、behavior_length=get_seq_length(codes)、distinct_codes=count_distinct(codes)、joined_window=zip_concat(slice_by_indices(codes, [0,2,4]), slice_by_indices(numbers, [0,2,4]), {"delimiter":"|"})。

两条路径：individual（每请求组一次 generate，共 8 次请求）与 grouped（一次 generateBatch）。引擎按输入载体分派（C10）：候选特征恒走原生 Batch kernel；纯共享序列特征在 individual 下走 Single kernel，在 grouped 下随共享值向量化走请求批域。

序列长度	individual median (ms)	grouped median (ms)	grouped/individual
50	58.147	24.207	2.40x
200	67.335	29.835	2.26x
1000	218.029	89.034	2.45x
3000	501.529	254.910	1.97x

诊断确认：表达式 DAG 共 34 逻辑/物理节点、10 个算子节点、无物理融合；individual 执行同时出现 SINGLE（纯共享特征）与 BATCH_NATIVE（候选特征）两类分派，grouped 执行 10 个算子节点全部 BATCH_NATIVE。

六、结论

- 批内复用的收益随序列长度放大：zip_concat / count_distinct / calc_delta_seq 等「每行可省 O(序列长度) 重算」的算子，在 L=3000 时加速比分别达到 510x / 164x / 75x；
- 轻计算算子（slice_by_indices / get_seq_length / log_base）批开销反噬，即使有复用也比 single 慢，应留给成本模型（C10）决定是否走 Batch；
- 无复用场景下 batchNative 全面回落到约 1x 或更差，其中 find_indices 因批内需为每个独立序列构建索引 map 而最慢（0.08x）；
- batchScalar（标量适配）恒在 1x 上下，是保底路径：无复用收益，也无显著损失；
- 双序列特征不改变结论：复用型算子收益更大（zip_concat 双序列拼接在 L=3000 达 713x，count_distinct 180x、calc_delta_seq 88x），轻计算算子依旧反噬；复用键覆盖多个序列特征工作正常，无复用场景回落规律不变；
- 特征表达式层（完整引擎链路）：grouped 批量请求比 individual 逐请求快约 2~2.4x，收益来自请求解码、调度与编码的摊销；候选特征两条路径都走原生 Batch kernel，纯共享特征在 individual 下走 Single kernel、在 grouped 下走请求批域。